LISTA DE EXERCÍCIOS DE REACT NATIVE – PRIMEIRA AVALIAÇÃO PARCIAL – PARTE I

2026.2

Questão 1 – Componente básico em React Native

Crie um componente funcional em React Native chamado `Apresentacao` que exiba as seguintes informações pessoais do aluno, uma em cada linha:

- Nome completo
- Curso da faculdade
- Cidade de origem

Requisitos:

1. Utilize apenas os componentes `View` e `Text`, importados de `react-native`.
2. Não aplique nenhum tipo de estilo (não use `style` nem `StyleSheet`).
3. Cada informação deve ser exibida em um componente `Text` próprio, todos dentro de um único `View`.
4. O componente deve ser exportado como padrão (`export default`).

Questão 2 – Modificações na Questão 1

Sobre o código da Questão 1, faça as seguintes alterações:

1. Utilize `StyleSheet` para centralizar os componentes na tela (utilize FlexBox).
2. Utilize `StyleSheet` para adcionar uma cor de fundo e mudar o tamanho das fontes.
3. Envolve a aplicação de uma forma que ela não “invada” a área dos botões do Android (`SafeAreaProvider, SafeAreaView`).

Questão 3 – Componentes e props em React Native

Crie um componente funcional (usando function) em React Native chamado `Disciplina` que receba, via `props`, os dados de uma disciplina e os exiba na tela, uma informação por linha:

- Nome da disciplina
- Quantidade de alunos
- Número do semestre
- Ementa (um breve texto descrevendo o conteúdo da disciplina)

Em seguida, crie um componente ascendente (pai), chamado `App`, que utilize o componente `Disciplina`, passando a ele, por meio de `props`, os dados de uma disciplina de sua escolha (por exemplo, com nome, quantidade de alunos, semestre e ementa preenchidos por você).

Requisitos:

1. Utilize apenas os componentes `View` e `Text`, importados de `react-native`.
2. Não aplique nenhum tipo de estilo (não use `style` nem `StyleSheet`).
3. O componente `Disciplina` deve receber os dados exclusivamente via `props`, sem valores fixos (*hardcoded*) dentro dele.
4. O componente `App` deve realizar a chamada `<Disciplina ... />`, informando todos os quatro dados como atributos.
5. Ambos os componentes devem estar em funções e o `App` deve ser exportado como padrão (`export default`).

Questão 4 – Composição de componentes e repasse de props em React Native

Crie três componentes funcionais em React Native, chamados `A`, `B` e `C`, seguindo a hierarquia descrita abaixo:

- O componente `A` renderiza internamente o componente `B`.

- O componente `B` renderiza internamente o componente `C`.
- O componente `A` define dois números (por exemplo, `5` e `8`) e os repassa ao componente `B` via `props`.
- O componente `B` não utiliza os números para nenhum cálculo: ele apenas repassa os mesmos dois valores ao componente `C`, também via `props`.
- O componente `C` recebe os dois números via `props`, calcula a soma entre eles e exibe o resultado na tela, **centralizado** (horizontal e verticalmente).

Requisitos:

1. Utilize os componentes `View` e `Text`, importados de `react-native`.
2. A centralização do resultado deve ser feita com estilo (`style` ou `StyleSheet`), usando as propriedades de layout *flexbox* adequadas (por exemplo, `flex`, `justifyContent` e `alignItems`).
3. O cálculo da soma deve ser realizado exclusivamente no componente `C`.
4. Os números não devem ser fixados (*hardcoded*) nos componentes `B` e `C`; eles só podem ser definidos no componente `A`.
5. O componente `A` deve ser exportado como padrão (`export default`).

Questão 5 – Composição de componentes, estado e entrada de dados em React Native

Crie três componentes funcionais em React Native, chamados `A`, `B` e `C`, seguindo a hierarquia descrita abaixo:

- O componente `A` renderiza internamente o componente `B`.
- O componente `B` renderiza internamente o componente `C`.
- O componente `A` implementa a entrada de dois números por meio de dois componentes `TextInput`, cujos valores devem ser armazenados em **variáveis de estado** (`useState`). Esses dois valores são repassados ao componente `B` via `props`.
- O componente `B` não utiliza os números para nenhum cálculo: ele apenas repassa os mesmos dois valores ao componente `C`, também via `props`.
- O componente `C` recebe os dois valores via `props`, calcula a soma entre eles e exibe o resultado na tela, **centralizado** (horizontal e verticalmente).

Requisitos:

1. Utilize os componentes `View`, `Text` e `TextInput`, importados de `react-native`, e o hook `useState`, importado de `react`.
2. Os `TextInput` devem aceitar **apenas entrada numérica**, utilizando uma abordagem adequada (por exemplo, a propriedade `keyboardType` com valor numérico e/ou o tratamento do texto digitado em `onChangeText` para remover caracteres não numéricos).
3. Os dois valores digitados devem ser mantidos em variáveis de estado no componente `A`, e o resultado exibido por `C` deve ser atualizado automaticamente conforme o usuário digita.
4. Como o `TextInput` trabalha com texto (*string*), os valores devem ser convertidos para número antes da soma. O componente deve tratar o caso de campo vazio (por exemplo, considerando o valor como `0`), evitando que o resultado exiba `NaN`.
5. A centralização do resultado deve ser feita com estilo (`style` ou `StyleSheet`), usando as propriedades de layout *flexbox* adequadas (por exemplo, `flex`, `justifyContent` e `alignItems`).
6. O cálculo da soma deve ser realizado exclusivamente no componente `C`.
7. Os números não devem ser fixados (*hardcoded*) nos componentes `B` e `C`; eles só podem ser originados da entrada de dados do componente `A`.
8. O componente `A` deve ser exportado como padrão (`export default`).

Questão 6 – Composição de componentes, estado e Context API em React Native

Crie três componentes funcionais em React Native, chamados `A`, `B` e `C`, seguindo a hierarquia descrita abaixo:

- O componente `A` renderiza internamente o componente `B`.
- O componente `B` renderiza internamente o componente `C`.
- O componente `A` implementa a entrada de dois números por meio de dois componentes `TextInput`, cujos valores devem ser armazenados em **variáveis de estado** (`useState`).
- Os dois valores **não** devem ser repassados via `props`. Em vez disso, o componente `A` deve disponibilizá-los por meio de um **Context** (criado com `createContext`), utilizando o `Provider` para envolver o componente `B`.
- O componente `B` não recebe nem repassa os números: ele apenas renderiza o componente `C`.
- O componente `C` acessa os dois valores diretamente a partir do Context (por meio do hook `useContext`), calcula a soma entre eles e exibe o resultado na tela, **centralizado** (horizontal e verticalmente).

Requisitos:

1. Utilize os componentes `View`, `Text` e `TextInput`, importados de `react-native`, e os recursos `useState`, `createContext` e `useContext`, importados de `react`.
2. O Context deve ser criado fora dos componentes (em escopo de módulo) e exportado ou compartilhado entre `A` e `C` no mesmo arquivo.
3. Os `TextInput` devem aceitar **apenas entrada numérica**, utilizando uma abordagem adequada (por exemplo, a propriedade `keyboardType` com valor numérico e/ou o tratamento do texto digitado em `onChangeText` para remover caracteres não numéricos).
4. Os dois valores digitados devem ser mantidos em variáveis de estado no componente `A`, e o resultado exibido por `C` deve ser atualizado automaticamente conforme o usuário digita.
5. Como o `TextInput` trabalha com texto (*string*), os valores devem ser convertidos para número antes da soma. O componente deve tratar o caso de campo vazio (por exemplo, considerando o valor como `0`), evitando que o resultado exiba `NaN`.
6. A centralização do resultado deve ser feita com estilo (`style` ou `StyleSheet`), usando as propriedades de layout *flexbox* adequadas (por exemplo, `flex`, `justifyContent` e `alignItems`).
7. O cálculo da soma deve ser realizado exclusivamente no componente `C`.
8. O componente `B` **não deve declarar nem receber props** relacionadas aos números, e os valores não devem ser fixados (*hardcoded*) em nenhum componente; eles só podem ser originados da entrada de dados do componente `A`.
9. O componente `A` deve ser exportado como padrão (`export default`).

Questão 7 – Variáveis de estado e eventos de clique em React Native

Crie um componente funcional em React Native chamado `Contador` que exiba um número na tela junto a um botão. Cada vez que o botão for pressionado, o valor exibido deve aumentar em uma unidade.

Requisitos:

1. Utilize os componentes `View`, `Text` e `Button`, importados de `react-native`, e o hook `useState`, importado de `react`.
2. O número deve ser armazenado em uma **variável de estado** (`useState`), com valor inicial `0`.
3. Ao pressionar o botão (propriedade `onPress`), o programa deve somar **uma unidade** ao valor atual do estado, utilizando a função de atualização do estado (não é permitido alterar a variável diretamente).
4. O valor atual do contador deve ser exibido na tela por meio de um componente `Text`, **junto ao botão**, e deve ser atualizado automaticamente a cada clique.
5. O número e o botão devem ficar **centralizados** (horizontal e verticalmente) na tela, utilizando estilo (`style` ou `StyleSheet`) com as propriedades de layout *flexbox* adequadas (por exemplo, `flex`, `justifyContent` e `alignItems`).

6. O botão deve exibir um título que indique sua função (por exemplo, "Somar 1").
7. O componente `Contador` deve ser exportado como padrão (`export default`).

Questão 8 (versão com useEffect) – Efeitos colaterais e variáveis de estado em React Native

Modifique o componente `Contador` da questão anterior para que ele passe a verificar, a cada alteração do contador, se o número atual é múltiplo de 5, exibindo o resultado na tela.

Requisitos:

1. Utilize os componentes `View`, `Text` e `Button`, importados de `react-native`, e os hooks `useState` e `useEffect`, importados de `react`.
2. Mantenha o comportamento anterior: o contador deve ser armazenado em uma **variável de estado** com valor inicial `0` e, ao pressionar o botão (por exemplo, com título "Somar 1"), deve ser incrementado em uma unidade por meio da função de atualização do estado.
3. Crie uma **segunda variável de estado** (por exemplo, `multiploDeCinco`) que armazene o resultado da verificação, com os valores `"VERDADEIRO"` ou `"FALSO"`.
4. Crie um efeito com `useEffect` que **observe a variável de estado do contador** (informando-a no array de dependências). Toda vez que o contador for modificado, a função dentro do `useEffect` deve verificar se o valor é múltiplo de 5 (por exemplo, usando o operador `%`) e atualizar a segunda variável de estado com `"VERDADEIRO"` ou `"FALSO"`.
5. A tela deve exibir, por meio de componentes `Text`, o valor atual do contador e, ao lado do rótulo "**MÚLTIPLO DE CINCO?**", o texto armazenado na segunda variável de estado (`VERDADEIRO` ou `FALSO`).
6. A verificação **não** deve ser feita diretamente no `onPress` do botão nem durante a renderização; ela deve ocorrer exclusivamente dentro do `useEffect`.
7. O conteúdo (contador, botão e verificação) deve ficar **centralizado** (horizontal e verticalmente) na tela, utilizando estilo (`style` ou `StyleSheet`) com as propriedades de layout *flexbox* adequadas (por exemplo, `flex`, `justifyContent` e `alignItems`).
8. O componente `Contador` deve ser exportado como padrão (`export default`).

Observação: considere o comportamento inicial do contador em `0`. Como `0` é múltiplo de 5, indique no código (ou em comentário) qual resultado a tela exibirá na primeira renderização.

Questão 9 – Navegação com Stack e passagem de parâmetros em React Native

Crie uma aplicação em React Native com navegação entre duas telas, representadas pelos componentes `A` e `B`, utilizando a biblioteca **React Navigation** com o navegador do tipo **Stack** (`@react-navigation/native` e `@react-navigation/native-stack`).

Componente `A` (tela de entrada de dados):

- Deve exibir três componentes `TextInput`, correspondentes aos campos **nome**, **sobrenome** e **idade**, e um botão com o título "**OK**".
- Os valores digitados devem ser armazenados em **variáveis de estado** (`useState`).
- Ao pressionar o botão "OK", o programa deve organizar os dados de entrada em um **objeto JSON** (por exemplo, `{ nome: "...", sobrenome: "...", idade: ... }`) e navegar para o componente `B`, enviando esse objeto como **parâmetro de navegação**.

Componente `B` (tela de exibição de dados):

- Deve receber o objeto enviado por `A` por meio dos parâmetros de navegação e exibir na tela, usando componentes `Text`, o **nome**, o **sobrenome** e a **idade** informados.

Requisitos:

1. Utilize os componentes `View`, `Text`, `TextInput` e `Button`, importados de `react-native`, e o hook `useState`, importado de `react`.
2. A navegação deve ser feita com o hook `useNavigation`, importado de `@react-navigation/native`, no componente `A`. A chamada de navegação deve enviar o objeto como parâmetro (por exemplo, `navigation.navigate("B", { dados })`).
3. Crie o navegador Stack com `createNativeStackNavigator`, registre os componentes `A` e `B` como telas (`Stack.Screen`) e envolva o navegador com o `NavigationContainer`. A tela `A` deve ser a tela inicial.
4. No componente `B`, os parâmetros recebidos devem ser lidos por meio da propriedade `route` (por exemplo, `route.params`) ou do hook `useRoute`.
5. O campo **idade** deve aceitar **apenas entrada numérica** (por exemplo, com `keyboardType` numérico e/ou tratamento do texto em `onChangeText`), e a idade deve ser convertida para número ao montar o objeto.
6. A montagem do objeto deve ocorrer exclusivamente no momento em que o botão "OK" for pressionado.
7. O componente que contém o `NavigationContainer` e o Stack deve ser exportado como padrão (`export default`).

Entrega: o código dos componentes `A` e `B` e do componente principal que configura a navegação (`NavigationContainer` e Stack).

Questão 10 – Listas com FlatList e variáveis de estado em React Native

Crie um componente funcional em React Native chamado `ListaLivros` que armazene, como **variável de estado**, uma lista (vetor) de objetos JSON, na qual cada objeto representa um livro com os campos **nome**, **autor** e **isbn**. O componente deve percorrer essa lista e exibir os dados de cada livro de forma estilizada, utilizando um `FlatList`.

Exemplo da estrutura de dados esperada:

```
{ nome: "Dom Casmurro", autor: "Machado de Assis", isbn: "978-85-359-0277-5" }
```

Requisitos:

1. Utilize os componentes `View`, `Text` e `FlatList`, importados de `react-native`, e o hook `useState`, importado de `react`.
2. A lista de livros deve ser armazenada em uma **variável de estado** (`useState`), inicializada com pelo menos **quatro livros** de sua escolha, cada um com os campos `nome`, `autor` e `isbn`.
3. A exibição da lista deve ser feita exclusivamente com o componente `FlatList`, utilizando as propriedades:
    - `data`, recebendo a variável de estado;
    - `renderItem`, responsável por renderizar os dados de cada livro;
    - `keyExtractor`, utilizando o `isbn` como chave única de cada item.
4. Cada item da lista deve ser apresentado em um "cartão" (`View`) contendo três componentes `Text`, um para cada campo (nome, autor e isbn), com rótulos que identifiquem cada informação (por exemplo, "Autor: ...", "ISBN: ...").
5. A apresentação deve ser **estilizada** por meio de `StyleSheet`, aplicando, no mínimo:
    - cor de fundo, borda arredondada (`borderRadius`) e espaçamentos (`padding` e `margin`) no cartão de cada livro;
    - destaque para o nome do livro (por exemplo, com `fontWeight` em negrito e tamanho de fonte maior que os demais campos);
    - cores ou tamanhos de fonte diferentes para o autor e o ISBN.
6. Não é permitido utilizar `map` ou laços de repetição para renderizar os itens; a iteração deve ser realizada pelo `FlatList`.
7. O componente `ListaLivros` deve ser exportado como padrão (`export default`).